

ArkanoidX

CRUD de Gestión de Perfiles de Usuario

Práctica: IA amb Desenvolupament Guiat per Especificació (OpenSpec)

Proyecto	ArkanoidX — Juego Arkanoid para móvil
Plataforma	Unity 2022 LTS — iOS / Android
Metodología	Spec-Driven Development (SDD) con IA
Herramienta IA	Claude (Anthropic)
Feature implementada	CRUD completo de UserProfile

1. Descripción de la Funcionalidad

ArkanoidX es una versión moderna del clásico juego arcade Arkanoid, desarrollada en Unity 2022 LTS para plataformas móviles (iOS y Android). El jugador controla una paleta para romper bloques con una pelota, con tres modos de juego disponibles: partida en solitario, multijugador en tiempo real con otro jugador mediante WebSocket, y enfrentamiento contra una inteligencia artificial.

La feature implementada es el **CRUD completo de perfiles de usuario**, que incluye las siguientes operaciones:

CREATE	Registro de nuevos jugadores con username, email y contraseña.
READ	Visualización del perfil propio con estadísticas de juego.
UPDATE	Edición del nombre de usuario y avatar seleccionable.
DELETE	Eliminación de cuenta con confirmación explícita.

La entidad central es **UserProfile**, con los campos: id (UUID), username, email, avatarIndex (0-7), y un sub-objeto stats con gamesPlayed, wins y highScore. La autenticación utiliza JWT Bearer Token guardado en PlayerPrefs.

La comunicación con el backend se realiza exclusivamente mediante **UnityWebRequest** (sin dependencias externas) a una API REST con los endpoints: POST /users, POST /auth/login, GET /users/me, PUT /users/me y DELETE /users/me.

2. Proceso Seguido con la IA

Se siguió una metodología **Spec-Driven Development (SDD)** estructurada en dos grandes bloques: la fase de especificación (opsx:propose) y la fase de implementación (opsx:apply). El agente de IA utilizado fue Claude de Anthropic.

2.1 Fase de Especificación (opsx:propose)

Se generaron tres documentos de especificación antes de escribir ninguna línea de código:

- **foundations.md**: Contexto del proyecto, objetivos, definición de la entidad UserProfile, restricciones técnicas (Unity 2022, C#, sin frameworks externos), actores del sistema y criterios de éxito medibles.
- **spec.md**: Comportamiento exacto de cada pantalla (AuthScreen, ProfileScreen), flujos de error y éxito para cada operación CRUD, contratos de API (endpoints, métodos, payloads JSON), validaciones en cliente y reglas de negocio.
- **plan.md**: Estructura de archivos del proyecto Unity, 4 fases de implementación con criterios de aceptación, prompt base para el agente y estrategia de corrección de desviaciones.

2.2 Fase de Implementación (opsx:apply)

La implementación se realizó en 4 fases iterativas, generando un archivo por prompt. El prompt base especificaba el archivo objetivo, los requisitos según el spec.md (con referencia explícita a la sección), y las restricciones técnicas.

Fase	Archivos generados	Prompts
Fase 1	UserProfile.cs, ApiService.cs, TokenManager.cs	3 (+1 corrección)
Fase 2	AuthService.cs, AuthScreen.cs	2 (+1 corrección)
Fase 3	UserService.cs, ProfileCache.cs, ProfileScreen.cs	3 (+1 corrección)
Fase 4	ConfirmDialog/Delete, UIManager.cs	2

3. Principales Problemas Encontrados

Problema 1 — API deprecated de Unity 2022 (ApiService.cs)

El agente generó código usando `www.isNetworkError`, que fue deprecado en Unity 2019.1. En Unity 2022 LTS la forma correcta es comprobar `www.result` contra los valores del enum `UnityWebRequest.Result`. Este error hubiera causado warnings en compilación y comportamiento impredecible en builds.

Solución: Prompt de corrección citando explícitamente la versión Unity 2022 y el enum correcto.

Problema 2 — Validación incompleta de username (AuthScreen.cs)

La validación del campo username solo comprobaba la longitud (3-20 caracteres) pero no los caracteres permitidos. Según spec.md sección 4, debe ser alfanumérico con underscore, sin espacios. El agente omitió la validación de caracteres especiales al no ser explícita en el prompt inicial.

Solución: Prompt de corrección con el regex exacto: `^[a-zA-Z0-9_]{3,20}$`

Problema 3 — Lógica offline incorrecta (ProfileScreen.cs)

El agente implementó la lógica offline como 'si la petición API falla, cargar desde caché'. Pero según spec.md sección 1.2, el comportamiento correcto es comprobar la conectividad ANTES de intentar la petición, usando `Application.internetReachability`. La diferencia es sutil pero importante: en el primer caso hay siempre una petición fallida y latencia; en el segundo, la UX es inmediata al detectar que no hay red.

Solución: Prompt de corrección citando la sección exacta del spec y describiendo el flujo de decisión correcto paso a paso.

4. Decisiones Tomadas

4.1 No modificar la especificación — solo los prompts

En ningún caso se modificó el spec.md para adaptarlo al código generado. La filosofía adoptada fue la inversa: si el agente se desviaba del spec, se corregía el agente mediante prompts más precisos. Esta decisión refuerza el valor del SDD y demuestra que la especificación era suficientemente clara y correcta desde el inicio.

4.2 Un archivo por prompt

Se decidió generar un archivo por llamada al agente, en lugar de pedir toda la implementación de una fase en un solo prompt. Esto facilitó la detección de errores, la corrección localizada y el control del proceso.

4.3 Referencia explícita al spec en cada prompt

Cada prompt de implementación incluía la referencia exacta a la sección del spec.md relevante (por ejemplo, 'según spec.md sección 1.2'). Esto redujo la ambigüedad y mejoró significativamente la tasa de acierto del agente en la primera iteración.

4.4 PlayerPrefs como fallback — sin backend obligatorio

Se decidió en plan.md incluir un fallback de caché local en PlayerPrefs para no bloquear el desarrollo por la no disponibilidad de un backend real. Esto permitió testear el flujo completo de la UI en el editor de Unity con datos mockeados.

5. Valoración Crítica

5.1 Lo que funcionó bien

- ✓ La especificación formal como base: tener spec.md como referencia evitó debates subjetivos sobre si el código era 'correcto'.
- ✓ La estructura en fases: implementar por fases con criterios de aceptación claros permitió verificar cada bloque antes de avanzar.
- ✓ Las correcciones fueron quirúrgicas: al citar la sección exacta del spec en los prompts de corrección, el agente modificó solo lo necesario sin romper el resto.
- ✓ Alta tasa de acierto en primera iteración: 7 de 10 archivos fueron correctos sin necesitar corrección.

5.2 Limitaciones detectadas en el agente IA

✗ Conocimiento de versión específica: el agente conoce Unity en general pero no siempre las diferencias entre versiones (el error de `isNetworkError` es un ejemplo claro).

✗ Inferencia de comportamiento implícito: cuando el spec describía el comportamiento offline, el agente infirió una implementación lógicamente plausible pero diferente a la especificada. Hay que ser más explícito.

✗ Validaciones de formulario: el agente tiende a implementar validaciones básicas (longitud) y omite las más específicas (regex) a menos que se indiquen explícitamente.

✗ Sin memoria entre prompts: cada prompt debe incluir el contexto necesario; el agente no recuerda decisiones tomadas en interacciones anteriores.

5.3 Conclusión general

El proceso SDD con IA demostró ser significativamente más controlable y predecible que el enfoque de 'prueba y error de prompts'. Tener una especificación formal como contrato entre el desarrollador y el agente permite identificar desviaciones rápidamente y corregirlas de forma precisa.

El agente IA es una herramienta potente para la generación de código estructural y repetitivo, pero requiere supervisión humana en los detalles técnicos específicos de plataforma y en los comportamientos condicionales complejos. El rol del desarrollador en SDD no es desaparecer, sino convertirse en **arquitecto de la especificación y auditor de la implementación**.